

This course has already ended.

« Round 15 (https://plus.cs.aalto.fi/studio_2/k2022/...) Chapter 15.2: Exceptions and attempts » (<https://pl...>)
CS-C2120 (https://plus.cs.aalto.fi/studio_2/k2022/) / Round 15 (https://plus.cs.aalto.fi/studio_2/k2022/w15/)
/ Chapter 15.1: More Tools to the Scala Toolbox

Chapter 15.1: More Tools to the Scala Toolbox

About this page:

Topics: Learn about case class and new ways of using match structure

What will I do? Primarily reading. Some exercises, too.

Indicative difficulty level: Intermediate

Rough estimate of workload? 2-3 hours.

Point value: S80

Related projects: Matchmaker 
(https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/projects/given/Matchmaker/).

What? No weekly summary?

The first round of exercises is still open and closes on Wednesday 27.1. We will publish the model solution to the UML design task as an additional chapter later.

Case class

The case class is a Scala construct that has a number of handy features. You could implement all of these features yourself, but just by adding the magic word "case", the compiler will automatically create a lot of boilerplate code, that is, code that is often needed and which the programmer would otherwise copy almost unchanged into the class.

Case classes facilitate, in particular, the creation of (immutable) data, the identification of complex structures, and access to class data.

```
// At its simplest a case class only includes
// a constructor definition. However,
// such a class already has many features.

case class Student(name: String, studentNumber: Int)

// ... of course, the class can also include some implementation

case class Teacher(name: String, course: Course) {
// ... class content ...
}
```

How do case classes differ from ordinary classes?

Let's explore in REPL

```
scala> case class Person(name: String, age: Int)
defined class Person
```

- A class companion object has method `apply` that automatically creates case-class entities (no `new` operator is required)

```
scala> val littleJohn = Person("John", 3)
littleJohn: Person = Person(John, 3)

scala> val bigJohn = Person("John", 4)
bigJohn: Person = Person(John, 4)
```

- All constructor parameters are automatically `val` type variables; they can be read without additional methods

```
scala> print(littleJohn.name)
John
```

- The class has a `copy` method that can make such a copy of its objects where some value of the original object has been changed but the others remain the same

```
scala> val newJohn = littleJohn.copy(age = 4)
newJohn: Person = Person(John, 4)
```

- The class automatically has methods `equals` and `hashCode` that can compare case-class objects to each other based on constructor parameters

```
scala> littleJohn == bigJohn
false

scala> newJohn == bigJohn
true
```

- The class has a method `toString` that prints the class name and constructor parameters values.

```
scala> print(littleJohn)
Person(John, 3)
```

- The class has a method `unapply` that, together with the `match` structure, provides a few handy features. These are discussed later in this chapter.
- There is a standard serialization code in the class that makes it easier to both store data and transfer it over the network.

Because the case class acts as a store for immutable data, it is often a good choice for storing and transmitting information in functional algorithms.

Match

The Match structure is a very versatile and expressive structure that one must be able to use. It is a selection structure that remotely resembles switch structures of C or Java, but is much more versatile.

Match takes a value as input and selects the first appropriate option based on the first appropriate pattern. In the example below, the variable `color` contains a string that is used to select between four options.

```
val inEnglish = color match {
  case "sininen" => "blue"
  case "punainen"=> "red"
  case "vihrea"  => "green"
  case unknownColor => "I don't know what " + unknownColor + " is."
}
```

If `color` contains the string "sininen", the statement mapped to the corresponding case is executed. In the above example, the match structure returns the value of the expression, but the different cases could also include executable code that only produces side effects and does not return anything.

The last case sentence deserves a little more attention. Unlike previous options, it does not define the pattern, but instead accepts all options. In addition, it defines the variable `unknownColor`, which can be used in the expression chosen by the case. The Match statement must always be able to choose one of the options, so the last option is usually the one that collects all inappropriate values for other options.

Here match made a selection based on the value, but it is especially handy when choosing values based on the value type.

match and case-classes

The Match Structure and Case Classes are designed to work together and they can make quite complex functionality with very little effort. As we saw in the previous section, case classes provide a quick way to define a set of types. In the example below, the match structure selects one of the alternatives based on the type of value contained in the variable. Note how in the pattern we store the value found into the variable teacher or student. The underscore `_` is a wildcard in the pattern, the value of which does not matter. In this example, it has provided a previously-mentioned default case that accepts invalid values for other options.

```
case class Course(courseName: String)

abstract class Person

case class Student(name: String, studentNumber: Int) extends Person
case class Teacher(name: String, course: Course) extends Person {
  // ...luokan sisältö...
}

def showPerson(person: Person): String = {
  person match {
    case teacher:Teacher => "Some Teacher"
    case student:Student => "A student called " + student.name
    case _                => "awkward... what's this?"
  }
}

val course = Course("Studio 2")
val teacher = Teacher("Lauri", course)
val oneStudent = Student("Eliza", 123456)
val otherStudent = Student("Pete", 222222)
```

Let us try:

```
scala> showPerson(teacher)
Some Teacher

scala> showPerson(oneStudent)
A student called Eliza
```

In the following example, you see how match compares the value with potentially recursive patterns, and initializes variables with the patterns it detects. The first case is selected if person contains a reference to the Teacher object. The name of the teacher is not required, so it has the previously mentioned wildcard. Instead, the course id (cid) is stored from the course taught by the teacher.

```
def showPersonMore(person: Person): String = {
  person match {
    case Teacher(_, Course(cid)) => "Someone who teaches " + cid
    case Student("Pete", _) => "It is Pete."
    case Student(name, number) => "Student number :" + number
    case _ => "awkward... what's this?"
  }
}
```

The second case sentence combines value and type identification in the same pattern. The constant value “Pete” within the pattern only selects students named Pete.

```
scala> showPersonMore(teacher)
Someone who teaches Studio 2

scala> showPersonMore(oneStudent)
Student number :123456

scala> showPersonMore(otherStudent)
It is Pete.
```

More examples

You can also identify tuples

```
val who = ("Richard", 45, true)

who match {
  case (name, age, dead) => s"$name, aged:$age is " + {if (dead) "dead" else "alive"}
  case _ => "awkward... what's this?"
}
```

A particularly handy match is with the Option Class

```
myOption match {
  case Some(job) => println("current job:" + job)
  case None =>      println("unemployed")
}
```

`_*` is the so-called sequence wildcard. It matches a sub-sequence (possibly the whole sequence).

```
myList match {
  case List("Pete", "Aki") => "The usual suspects"
  case List("Pete")       => "alone"
  case List("Pete", _ *)  => "Pete's friends"
  case _ =>
}
```

warning: match is not exhaustive

If it is possible that the value of the variable is not suitable for any option, and the default case (wildcard `_` or other) cannot be found, the compiler issues an error message. When comparing values or structures, it is usual to add a default case.

If we have a match structure where (case) classes or traits make up all possible alternatives, we can sometimes report in the class / trait from which the alternatives have been inherited that it is a so-called sealed class / trait. All sub-classes must then be presented in the same file. In this case, the compiler is already familiar with all suitable values (subclasses).

```
sealed trait Directions
case class Left() extends Directions
case class Right() extends Directions
```

Extractor

How does match get constructor parameters from objects? Could this also be used with other classes?

The answer is yes, and the solution is found in a companion object where an `unapply` method can be implemented. This method takes the object as a parameter and, if successful, returns the values found in a tuple contained in an `Option` wrap. By implementing the method yourself, you allow for objects of other classes to be applied to the match structure.

```
class Student(val name: String, val number: Int)

object Student {
  def unapply(s: Student): Option[(String, Int)] =
    Some((s.name, s.number))
}

val guy = new Student("james", 314)

guy match {
  case Student(name, number) => println(number + ":" + name)
}
```

Keyword / standard arguments

It is often possible to define method and constructor parameters that have a certain default value and exceptions to this are rare. This is useful especially with library functions that have a huge number of parameters that are often not needed.

When a method / constructor with default values is called, just the ordinary parameters can be provided, while the variables corresponding to the standard arguments will be filled automatically. See the first *doSomething* call below.

If you want to replace the value of a certain default valued parameter, you can specify it in a call. See the lower *doSomething* call.

```
def doSomething( name: String,
                 age : Int = 18,
                 job : String = "student" ) = {
  println(name + ", aged " + age + ", is a " + job)
}

doSomething("Petteri")

// prints: "Petteri, aged 18, is a student"

doSomething("Petteri", job="taxi driver")

// prints: "Petteri, aged 18, is a taxi driver"
```

For comprehensions

In Scala, *for* is a very expressive structure. So far, it has been used to produce only one loop. However, the *for* structure can include several nested loops, which is particularly useful when dealing with, for example, a list of Option Items. (Option is also a collection, albeit small).

There are a few differences in the syntax:

- the generator(s) are listed in the braces as a difference from the normal for-loop
- The loop code is immediately after the braces

In the example below, the strings in the Option items in the Buffer object are printed. Because Some is a single-sized collection and a None zero-size collection, they work brilliantly in the for-structure.

```
val possibleStrings: Buffer [Option [String]] = ... something ...

for {
  listItem <- possibleStrings
  oneString <- listItem
} println(oneString)
```

yield

For-comprehension can, instead of executing code within the loop, produce a new collection structure. Instead of the executable command, write the word *yield* and then the expression that generates an item for the individual collection.

```

val numberedAnimals = for {
  a <- (1 to 5)
  b <- List("cat", "dog")
} yield b + " " + a

// Vector("cat 1", "dog 1", "cat 2", "dog 2", ...)

val characters = for {
  item <- numberedAnimals
  c <- item
} yield c

// Vector(c, a, t, , 1, d, o, g, , 1, c, a, t, , 2, d, o, g, , 2, c, a, t, , 3, ...)

```

Small task

Inspect the short function below, namely its output based in different input values.

```

case class Person(name:String, age:Int=5)
case class Student(name:String, age:Int)

def nameIt(it:Any) = {
  println(it match {
    case Person(name, age) => s"It's $name($age)"
    case (name, age) => s"Sure, it's $name($age)"
    case _ => "Eh.."
  })
}

```

 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Exercise 1

This course has been archived (Thursday, 1 June 2023, 12:00).

Question 1 2 / 2

nameIt(Person("Matti", 5))

Show anyway

- ☒ It's Matti (5)
- ☐ Sure, it's Matti (5)
- ☐ Eh..

✓ Correct!

Question 2 2 / 2

nameIt(Student("Mikko", 21))

- ☐ It's Mikko (21)
- ☐ Sure, it's Mikko (21)
- ☒ Eh..

✓ Correct!

Question 3 2 / 2

```
nameIt(("Mikko", 21))
```

- ☐ It's Mikko (21)
- ☒ **Sure, it's Mikko (21)**
- ☐ Eh..

✓ Correct!

Question 4 2 / 2

```
nameIt(("Matti"))
```

- ☐ It's Matti (None)
- ☐ Sure, it's Matti (None)
- ☒ **Eh..**

✓ Correct!

Question 5 2 / 2

```
nameIt(Person("Mauri"))
```

- ☐ It's Mauri (None)
- ☒ **It's Mauri (5)**
- ☐ Eh..

✓ Correct!

Submit

MatchMaker 70p

In the next exercise, your objective is to practice using the match-case structure by creating a simple Matchmaker program for finding two similar people. The program considers people who have different interests and the program tries to find a suitable match with similar interests. Similarity of a pair is represented by a match score that the program calculates.

The people in this exercise are represented by the following class structure:

```
Person(name: Name, favoriteColor: Color, hobby: String, profession: String)
```

There are also different Person types that affect what the person is looking for in their partner. The person types are subclasses of the Person class and they describe what kind of partner the person is looking for:

ColorNeutral– The person does not care about favorite colors. The person is satisfied if at least one point of interest matches.

Hobbyist – The person's only interest is that their partner has the same hobby as him/herself. If the other person is also a Hobbyist, they get extra points.

Perfectionist – The person wants their partner to have exactly the same interests as themselves. If the other person is also a Perfectionist, they get full points.

Objectives of the exercise:

- For each of the sub classes of the Person class, implement the method `calculateMatch(other: Person)` that calculates a match score for this person and another person.
- For the Person class, implement the method `bothMatch` that compares a match score for two people from both points of view and calculates the average of their scores.
- Code the method `matchMap` for Matchmaker that maps each person to a list of mutual match scores between each other person in the system paired with the person's own information. The item with the highest score on the list would be the best matches for that person.

 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Exercise 2 (MatchMaker)

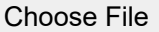
Select your files for grading

This course has been archived (Thursday, 1 June 2023, 12:00).

 **Matchmaker.scala**  Show

 Choose File No file chosen

 **Person.scala**

 Choose File No file chosen

 Submit

Summary

- We learned about case classes and the *match* structure and how they could be used.
- We learned that the *for* statement has features which were not discussed earlier.
- We also learned about some other Scala structures which could be useful.

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately

Loading assignment...

« Round 15 (https://plus.cs.aalto.fi/studio_2/k2022/...

Chapter 15.2: Exceptions and attempts » (<https://pl...>